TC-ViTok: Task-Conditioned Visual Tokenization with Informative Token Merge and Adaptive Budget

https://github.com/whyeventry0103/TC-ViTok

Muhammad Ali Shah (27100172) Talha Hassan Ul Haq (27100306)

Abstract

Vision-language models (VLMs) such as LLaVA-1.5 encode every image into a fixed 576 visual tokens regardless of question complexity, creating a quadratic attention bottleneck dominated by uninformative regions. We present **TC-ViTok**, a query-conditioned visual tokenizer that selects k=144 task-relevant tokens *before* the language model using a lightweight (≈887K-parameter) policy network operating on frozen pre-computed features. Three technical contributions enable end-to-end training despite discrete selection: (i) a Gumbel-Sigmoid straight-through estimator with a novel *tau-independent additive gradient correction* that maintains learning signal as temperature anneals; (ii) *Informative Token Merge*, a parameter-free mechanism that absorbs discarded background tokens into the selected set via TPN-score-weighted cosine assignment; and (iii) a *BudgetPredictor* (≈25K params) with a differentiable threshold trick for per-sample variable token counts.

On LLaVA-1.5-7B at $4\times$ token compression, TC-ViTok achieves **67.12% on GQA val_balanced**, **exceeding the uncompressed 576-token model (61.0%) by +6.1 points**. The model also reaches 72.27% on VQAv2 and 79.09% POPE F1, with a $3.5\times$ **theoretical reduction in LLM prefill FLOPs** and 71% smaller KV cache. The GQA improvement —- the only metric where compression *outperforms* the uncompressed baseline —- demonstrates that for compositional spatial reasoning, query-conditioned selection is not merely a compression technique but an active denoising mechanism. We further document the failure modes of eight earlier trainable-pooler variants, providing concrete guidance for future modular visual token systems. Code: https://github.com/whyeventry0103/TC-ViTok.

1. Introduction

Modern VLMs such as LLaVA-1.5 (Liu et al., 2023) attach a large language model (LLM) to a frozen CLIP ViT-L/14 vision encoder via a learned multimodal projector. At inference, the encoder maps a 336×336 image to 576 patch tokens (24×24 grid, patch size 14×14), which are prepended to the tokenized question and fed into the LLM. Since attention cost is quadratic in sequence length, these 576 visual tokens (which far outnumber the $\sim$30 typical question tokens) dominate prefill FLOPs.

This fixed-token representation is informationally wasteful. A question asking "What is written on the sign?" requires only the small image region containing text, yet the LLM still receives all 576 tokens including sky, background, and irrelevant objects. Conversely, "Is there a person in the image?" needs a global scan rather than fine-grained local detail. The token budget is determined by image resolution, not by what the question requires.

Prior work has addressed the redundancy half of this problem through token compression: Token Merging (ToMe) (Bolya et al., 2023) and EViT (Liang et al., 2022) reduce tokens within the ViT; FastV (Chen et al., 2024) and LLaVA-PruMerge (Shang et al., 2024) prune after encoding; SparseVLM (Zhang et al., 2025) recycles pruned tokens inside the LLM. However, all these methods decide which tokens to keep based on *image content alone* — the user's question plays no role. This is the query-conditioning gap.

The few approaches that incorporate query signals do so either inside the LLM's attention layers (SparseVLM, FocusLLaVA (Zhu et al., 2024)), which still incurs the full early-layer LLM cost, or inside the vision encoder (QG-VTC (Li et al., 2025)), which tightly couples the compression logic to a specific encoder architecture and sacrifices modularity. We are unaware of any method that provides query-conditioned token selection as a *separate, lightweight module* operating on pre-computed frozen features *before* the LLM.

We make four contributions:

(1) A modular, query-conditioned token-selection architec-

ture that operates on frozen pre-computed features *before* the LLM, decoupling compression from both encoder and LLM internals (Section 3.3).

(2) A novel *tau-independent additive gradient correction* for Gumbel-Sigmoid STE that maintains a non-zero, dot-product gradient signal at any temperature, eliminating the gradient-death failure mode standard STE exhibits when annealing toward hard selection (Section 3.4, Eq. 7).

(3) *Informative Token Merge*: a zero-parameter mechanism that recovers near-threshold context tokens via TPN-score-weighted cosine assignment, explicitly avoiding the trainable-pooler co-adaptation pathology we documented across eight failed experiments (Section 3.5).

(4) A *BudgetPredictor* with a *differentiable threshold trick* (Eq. 12) that routes budget-penalty gradients through what would otherwise be a non-differentiable per-sample top-k operation (Section 3.6).

Empirically, our system achieves **67.12% on GQA val_balanced —** $+6.1$ **points above full LLaVA-1.5-7B** – at $4\times$ token compression (k=144) and a $3.4\times$ theoretical LLM-prefill speedup. The GQA improvement, surprising at first reading, follows directly from query-conditioned selection acting as an active denoiser: GQA's compositional structure rewards precision over coverage, a regime where fewer better-chosen tokens beat all 576.

2. Related Work

Query-agnostic visual token compression. ToMe (Bolya et al., 2023) merges visually similar tokens via bipartite matching inside ViT layers, achieving $2\times$ throughput at modest accuracy loss on classification. DynamicViT (Rao et al., 2021) learns binary dropping masks at intermediate layers via Gumbel-Softmax, trained with distillation. EViT (Liang et al., 2022) uses CLS-token attention to rank tokens and fuses low-importance ones. Post-encoder methods include LLaVA-PruMerge (Shang et al., 2024), which prunes by CLS-attention rank then merges pruned tokens with nearest retained neighbors via key-similarity weighting. FastV (Chen et al., 2024) observes that LLM text-to-visual attention collapses after layer 2 and prunes low-attention tokens at that boundary. All these methods share a fundamental limitation: the compression decision is made without knowledge of the query, systematically harming spatially-grounded tasks (localization, OCR) more than global tasks (Chen et al., 2024; Endo et al., 2025).

Query-aware compression. BLIP-2 (Li et al., 2023a)'s Q-Former extracts visual tokens via cross-attention with learned query embeddings, but the queries are fixed parameters shared across all inputs rather than derived from the user's question. SparseVLM (Zhang et al., 2025) incorporates text guidance through the LLM's own text-to-visual attention with adaptive per-layer recycling, but selection happens inside the LLM after all tokens have already been injected. QG-VTC (Li et al., 2025) achieves near-uncompressed performance with 1/8 of tokens by embedding the question into the ViT's feature space and progressively compressing within the encoder — but this couples compression to the encoder's internal architecture. FocusLLaVA (Zhu et al., 2024) applies coarse-then-fine instruction-guided selection, but the text-guided stage remains inside the LLM. TC-ViTok closes this gap by implementing query-conditioning as a modular pre-LLM module that operates on frozen, pre-computed features from both encoders.

Table 1. Positioning of TC-ViTok in the visual token compression landscape. Pre-LLM = decision made before LLM forward pass (saves all LLM FLOPs); inside LLM = decision made between LLM layers (saves only late-layer FLOPs).

Method	Where	Query	Recover	Adapt k
ToMe (Bolya et al., 2023)	in-encoder	✗	merge	✗
DynamicViT (Rao et al., 2021)	in-encoder	✗	✗	✗
EViT (Liang et al., 2022)	in-encoder	✗	fuse	✗
PruMerge (Shang et al., 2024)	post-encoder	✗	merge	✗
FastV (Chen et al., 2024)	in-LLM (L=2)	implicit	✗	✗
SparseVLM (Zhang et al., 2025)	in-LLM	via attn	recycle	per-layer
QG-VTC (Li et al., 2025)	in-encoder	explicit	merge	✗
FocusLLaVA (Zhu et al., 2024)	in-LLM	explicit	coarse-fine	✗
TC-ViTok (ours)	**pre-LLM**	**explicit**	**merge**	**per-sample**

Differentiable discrete optimization. The Gumbel-Softmax trick (Jang et al., 2017; Maddison et al., 2017) enables gradient estimation through categorical variables via a continuous relaxation parameterized by temperature. The straight-through estimator (STE) (Bengio et al., 2013) uses the hard binary mask in the forward pass while routing gradients through the soft relaxation, providing low-variance gradient estimates for subset selection. AdaViT (Meng et al., 2022) demonstrates learned per-sample computation budgets in ViTs. TC-ViTok extends these ideas with (i) Gumbel-Sigmoid for independent binary decisions across 576 positions, and (ii) an additive tau-independent gradient correction that resolves a gradient vanishing failure mode at low temperature absent from prior work.

Table 1 positions TC-ViTok in this landscape along four operationally-relevant axes: *when* compression occurs, *whether* the question conditions selection, *whether* discarded tokens are recovered, and *whether* the budget can vary per sample.

To our knowledge, TC-ViTok is the first method that combines (a) explicit question-conditioned selection, (b) pre-LLM placement on frozen features, and (c) per-sample adaptive budget into a single modular system.

3. Methodology

3.1. Problem Formulation

Given an image $\mathbf{I}$ and a natural-language question $\mathbf{Q}$, a VLM f_θ generates an answer $\mathbf{A}$. Standard LLaVA-1.5 encodes $\mathbf{I}$ into $N = 576$ patch tokens $\mathbf{v}_f \in \mathbb{R}^{N \times D_v}$ (where $D_v = 1024$) and processes all of them. We instead learn a policy:

$$\pi_\phi(\mathbf{I}, \mathbf{Q}) \;\to\; \mathcal{S} \subseteq \{1, \ldots, N\}, \quad |\mathcal{S}| = k$$

where $k \ll N$ and ϕ is trained end-to-end from the VQA loss. The LLM receives only the k selected tokens, directly reducing prefill FLOPs.

3.2. Frozen Encoders

We use three frozen pre-trained encoders. **DINOv2-S** (Oquab et al., 2023) encodes a 98×98 crop of $\mathbf{I}$ (7 patches per side) to produce a coarse spatial map $\mathbf{v}_c \in \mathbb{R}^{49 \times 384}$. We use 98px input (not the standard 518px) to give the TPN a cheap, 7×7 spatial grid that is efficient to cross-attend. DINOv2's self-supervised training preserves patch-level spatial locality better than CLIP's contrastive objective, making it a more precise spatial guide. **CLIP ViT-B/32** (Radford et al., 2021) encodes $\mathbf{Q}$ to produce a query embedding $\mathbf{q} \in \mathbb{R}^{512}$ (the EOS token representation). **CLIP ViT-L/14** (Radford et al., 2021) encodes a 336×336 crop to produce full-resolution features $\mathbf{v}_f \in \mathbb{R}^{576 \times 1024}$, which are already aligned to LLaVA-1.5's LLM since LLaVA uses the same encoder. All three encoders are always frozen — no gradients flow into them.

3.3. Tokenization Policy Network (TPN)

The TPN (≈887K parameters) maps $(\mathbf{v}_c, \mathbf{q})$ to importance scores $\mathbf{s}_N \in \mathbb{R}^N$ and selection indices $\text{idx} \in \mathbb{Z}^k$.

Cross-attention (4 heads). The question embedding is treated as a single query:

$$Q = \mathbf{q}\, W_Q \in \mathbb{R}^{1 \times 384}, \quad K = \mathbf{v}_c\, W_K \in \mathbb{R}^{M \times 384}, \quad V = \mathbf{v}_c\, W_V \in \mathbb{R}^{M \times 384}$$

where $M = 49$. Multi-head attention yields an attended context vector $\text{ctx} \in \mathbb{R}^{384}$ and head-averaged spatial weights $\boldsymbol{\alpha} \in \mathbb{R}^M$. The context encodes *what* visual content is relevant to the question; $\boldsymbol{\alpha}$ encodes *where*.

Score MLP. $[\text{ctx};\, \boldsymbol{\alpha}] \in \mathbb{R}^{433}$ is passed through a LayerNorm and two-layer MLP to produce 49 coarse scores, then bilinearly upsampled $7 \times 7 \to 24 \times 24$ to yield $\mathbf{s}_N \in \mathbb{R}^{576}$.

Instance normalization. $\mathbf{s}_N$ is normalized to zero mean and unit variance per sample: $\mathbf{s}_N \leftarrow (\mathbf{s}_N - \mu)/\sigma$. Without this, Adam pushes score magnitudes to ± 100 within $\sim$1000 steps. At that scale, Gumbel noise (magnitude $\approx \tau \approx 1.0$) is negligible compared to scores, and the same 576 positions are always selected regardless of question — entropy collapse. Alternative normalizations (tanh, batch norm) also fail: tanh saturates at ± 1 ($|\tanh'(\pm 100)| \approx 10^{-87}$), and batch norm carries a stateful running mean that disrupts inference. Instance norm permanently anchors scores to $\mu = 0, \sigma = 1$ with no learnable parameters the optimizer can circumvent.

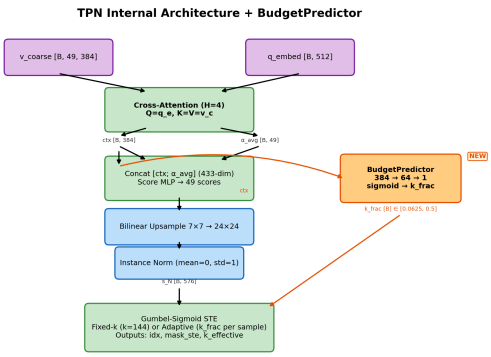


Figure 1. **TPN internal architecture and BudgetPredictor wiring.** Cross-attention with $H{=}4$ heads ($Q{=}\mathbf{q}$, $K{=}V{=}\mathbf{v}_c$) produces a context vector $\text{ctx}{\in}\mathbb{R}^{384}$ and head-averaged spatial weights $\boldsymbol{\alpha}{\in}\mathbb{R}^{49}$. The $[\text{ctx};\boldsymbol{\alpha}]$ concatenation passes through a Score MLP producing 49 coarse scores, which are bilinearly upsampled $7{\times}7 \to 24{\times}24$ and instance-normalized to yield $\mathbf{s}_N{\in}\mathbb{R}^{576}$. Phase 1 Adaptive shares ctx with the BudgetPredictor (orange) which emits the per-sample token fraction $k_{\text{frac}}{\in}[0.0625, 0.5]$ consumed by the Adaptive Gumbel-STE.

3.4. Gumbel-Sigmoid Straight-Through Estimator

Token selection is a discrete top-k operation with zero gradients almost everywhere. We adapt the Gumbel-Sigmoid STE (Jang et al., 2017; Bengio et al., 2013) for independent binary subset selection over 576 positions.

Forward pass. Gumbel noise $\mathbf{g} \sim \text{Gumbel}(0, 1)$ provides per-sample diversity; the temperature τ controls the soft-to-hard transition:

$$\mathbf{n} = \mathbf{s}_N + \mathbf{g}, \tag{1}$$

$$\mathbf{m}_{\text{soft}} = \sigma(\mathbf{n}/\tau) \in (0, 1)^N, \tag{2}$$

$$\text{idx} = \text{topk}(\mathbf{n},\, k), \tag{3}$$

$$\mathbf{m}_{\text{hard}} = \mathbf{0};\;\; \mathbf{m}_{\text{hard}}[\text{idx}] = 1, \tag{4}$$

$$\mathbf{m}_{\text{STE}} = \mathbf{m}_{\text{hard}} - \mathbf{m}_{\text{soft}}.\text{detach}() + \mathbf{m}_{\text{soft}}. \tag{5}$$

In the forward pass, $\mathbf{m}_{\text{STE}} = \mathbf{m}_{\text{hard}}$ (the $\mathbf{m}_{\text{soft}}$ terms cancel). In the backward pass, the gradient flows through $\mathbf{m}_{\text{soft}}$. Noise in Eq. (1) (not raw scores) is used for top-k to ensure per-sample selection diversity across the batch; using raw scores selects identical positions for all samples, collapsing entropy. Temperature is annealed linearly $\tau : 1.0 \to 0.3$ over training.

Tau-independent additive gradient correction. At low τ, $\mathbf{m}_{\text{soft}}$ saturates toward $\{0, 1\}$, making $\partial \mathbf{m}_{\text{soft}} / \partial \mathbf{s}_N \approx 0$ for all positions with $|\mathbf{s}_N[i]| > 1$. This kills the gradient path to TPN weights, halting learning after τ has annealed. We resolve this by multiplying gathered visual tokens by an additive correction:

$$\mathbf{v}_{sel} = \text{gather}(\mathbf{v}_f, \text{idx}), \quad \mathbf{c} = \text{gather}(\mathbf{s}_N, \text{idx}), \tag{6}$$

$$\mathbf{v}_{sel} \leftarrow \mathbf{v}_{sel} \odot (1 + \mathbf{c} - \mathbf{c}.\text{detach}()). \tag{7}$$

In the forward pass, $\mathbf{c} - \mathbf{c}.\text{detach}() = \mathbf{0}$, so $\mathbf{v}_{sel}$ is unchanged. In the backward pass, for selected position j:

$$\frac{\partial \mathcal{L}_{\text{VQA}}}{\partial \mathbf{s}_N[j]} = \sum_{d=1}^{D_v} \frac{\partial \mathcal{L}}{\partial \mathbf{v}_{sel}[j, d]} \cdot \mathbf{v}_{sel}[j, d],$$

a dot product between the VQA gradient and visual features at position j. This gradient is non-zero at any τ and provides a dense, token-level supervision signal for TPN throughout training.

3.5. Informative Token Merge

Hard dropping of the 432 unselected tokens discards all the contextual information they carry. PruMerge (Shang et al., 2024) shows that merging such tokens recovers $\sim$2% VQAv2 over pure pruning at the same budget. We implement a query-conditioned variant: the TPN score of an unselected token determines how strongly it contributes to the merge, so tokens that were "almost selected" (near-threshold, task-relevant) contribute more than distant background tokens.

Let $\mathbf{v}_{sel} \in \mathbb{R}^{k \times D_v}$ be the gathered selected tokens and $\mathbf{v}_{unsel} \in \mathbb{R}^{(N-k) \times D_v}$ the unselected tokens. Each unselected token i is soft-assigned to the nearest selected token by cosine similarity:

$$A_{ij} = \text{softmax}_j \left(\frac{\hat{\mathbf{v}}_{\text{unsel},i}^{\top} \hat{\mathbf{v}}_{\text{sel},j}}{\tau_m} \right), \tag{8}$$

where $\hat{\cdot}$ denotes ℓ_2-normalization and τ_m=0.1. Each unselected token is further weighted by its TPN score:

$$w_i = \text{softmax}_i(s_i^{\text{unsel}}), \tag{9}$$

where $s_i^{\text{unsel}} = \mathbf{s}_N[\text{unsel_idx}[i]]$. The normalized contribution matrix $\tilde{A}_{ij} = w_i A_{ij} / \sum_{i'} w_{i'} A_{i'j}$ aggregates unselected content into each selected token:

$$\mathbf{v}_{merged} = \mathbf{v}_{sel} + \beta \cdot \tilde{A}^{\top} \mathbf{v}_{unsel}, \quad \beta = 0.3. \tag{10}$$

This mechanism introduces zero new parameters, eliminates any co-adaptation risk with LoRA, and preserves the exact token count k between training and evaluation (avoiding the train-eval mismatch that corrupted our earlier trainable pooler experiments).

3.6. Adaptive Budget

The fixed k=144 model compresses all images equally. We extend TC-ViTok with a *BudgetPredictor* that learns per-sample token fractions.

BudgetPredictor architecture (≈25K params).

$$k_{\text{frac}} = \sigma\big(\text{Linear}_{64 \to 1}\big(\text{GELU}\big(\text{Linear}_{385 \to 64}([\text{ctx}; \bar{h}(\mathbf{s}_N)])\big)\big)\big), \tag{11}$$

where $\text{ctx} \in \mathbb{R}^{384}$ is the TPN's attended visual context vector (encoding which visual regions are relevant to the question), $\bar{h}(\mathbf{s}_N) \in \mathbb{R}^1$ is the normalized entropy of $\mathbf{s}_N$ (a measure of how focused the importance distribution is), and σ clips the output to $[k_{\min}/N, k_{\max}/N] = [36/576, 288/576]$.

Differentiable threshold. Converting k_{frac} to a hard selection requires a discrete step: $k_{\text{hard}} = \lfloor k_{\text{frac}} \cdot N \rceil$. To route gradients from the budget penalty to the predictor despite this non-differentiability, we define a modified threshold:

$$\theta = \theta_{\text{hard}} - (k_{\text{target}} - k_{\text{target}}.\text{detach}()), \tag{12}$$

where $k_{\text{target}} = k_{\text{frac}} \cdot N$ and θ_{hard} is the score at position k_{hard} in the sorted score sequence (no gradient). In the forward pass, Eq. (12) evaluates to θ_{hard} (the correction is zero). In the backward pass, $\partial \theta / \partial k_{\text{frac}} = -N$, providing a non-zero gradient to the BudgetPredictor from the budget penalty loss. The shifted soft mask $\sigma((\mathbf{n} - \theta)/\tau)$ produces a differentiable effective count $k_{\text{eff}} = \sum_i m_{\text{soft},i}$ used in the loss; hard selection uses per-sample $\text{topk}(k_{\text{hard}})$.

Budget penalty loss. $\mathcal{L}_{\text{budget}} = \lambda_b \cdot \mathbb{E}[k_{\text{eff}}/N]$ penalizes using more tokens. Higher λ_b forces fewer tokens but risks accuracy degradation. We sweep $\lambda_b \in \{0.05, 0.1, 0.2, 0.4\}$ to trace the Pareto frontier.

3.7. Loss Functions

The total training loss is:

$$\mathcal{L} = \mathcal{L}_{\text{VQA}} + \lambda_H(-H) + \lambda_d \mathcal{L}_{\text{div}} + \lambda_b \mathcal{L}_{\text{budget}}, \tag{13}$$

where $\mathcal{L}_{\text{VQA}}$ is the next-token prediction cross-entropy on answer tokens (labels $= -100$ for visual/question tokens), H is the spatial entropy of the batch-mean selection distribution, and $\mathcal{L}_{\text{div}}$ is a diversity loss. Table 2 summarizes each term.

Spatial entropy ($-H$). Without this term, TPN quickly focuses all selections on a fixed spatial region (the image center, which is most discriminative in VQAv2). $H = -\sum_i p_i \log p_i$ where $\mathbf{p}$ is the normalized batch-mean selection frequency; minimizing $-H$ maximizes distributional spread across positions.

Diversity loss ($\mathcal{L}_{\text{div}}$). Paired adjacent batch samples: $\mathcal{L}_{\text{div}} = \mathbb{E}[\cos(\mathbf{s}_{Na}, \mathbf{s}_{Nb}) \cdot (1 - \cos(\mathbf{q}_a, \mathbf{q}_b))]$. This penal-

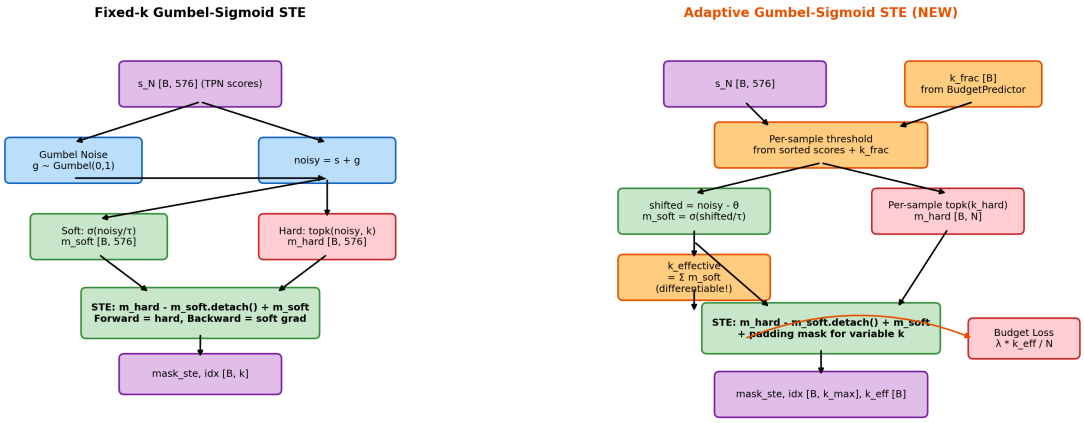


Figure 2. **Fixed-k vs. Adaptive Gumbel-Sigmoid STE** (Phase 1 Final vs. Phase 1 Adaptive). *Left:* the standard fixed-budget pipeline: scores + Gumbel noise feed both a soft sigmoid (for backward pass) and a hard top-k=144 selection (for forward pass), composed via the STE identity $\mathbf{m}_{\text{STE}}=\mathbf{m}_{\text{hard}}-\mathbf{m}_{\text{soft}}.\text{detach}()+\mathbf{m}_{\text{soft}}$. *Right:* the adaptive variant inserts a per-sample threshold θ derived from the BudgetPredictor's k_{frac} output. The shifted soft mask $\sigma((\mathbf{n}-\theta)/\tau)$ produces a differentiable effective count $k_{\text{eff}}=\sum_i m_{\text{soft},i}$ that drives the budget loss; per-sample top-k_{hard} provides the hard mask with right-padding to $k_{\max}$ for batched processing.

Table 2. Loss function components.

Term	Purpose	Weight
$\mathcal{L}_{\text{VQA}}$	NTP cross-entropy on answer tokens	1.0
$-H$	Prevent spatial collapse of TPN	0.02
$\mathcal{L}_{\text{div}}$	Prevent query-ignoring	0.1
$\mathcal{L}_{\text{budget}}$	Penalize token usage (Phase 1-Adap.)	sweep

izes similar selections for samples with dissimilar questions, pushing TPN to route differently depending on the query.

3.8. Training Protocol

Training follows a three-phase curriculum where each phase adds one component.

Phase 0 — TPN Warmup (10K steps). Only the TPN is trained; LLaVA's LLM, MM projector, and LoRA are frozen. With LoRA frozen, the only way to reduce $\mathcal{L}_{\text{VQA}}$ is through better token selection — forcing TPN to learn meaningful query-conditioned scores before joint co-training. Temperature $\tau = 1.0$ (fixed); data: VQAv2 training split.

Phase 1 Final — Joint Training with Token Merge (80K steps). TPN + LoRA (rank 64) + MM projector are co-trained. LoRA is frozen for the first 3000 steps (*warm-freeze*) to prevent it from immediately compensating for TPN imperfections, preserving the gradient signal that forces TPN to improve. τ anneals $1.0 \rightarrow 0.3$; data: VQAv2 + GQA; effective batch size 128 ($4 \times 16 \times 2$ GPUs). Checkpoint loaded from Phase 0.

Table 3. Training hyperparameters per phase. P1-F = Phase 1 Final; P1-A = Phase 1 Adaptive. The expanded LoRA target set in P1-F (FFN layers) is critical — it allows the LLM to adapt to merged token representations.

Hyperparameter	Phase 0	Phase 1-F	Phase 1-A
Steps	10K	80K	40K
LR (TPN)	1e−3	3e−3	1e−3
LR (LoRA)	—	1e−4	2e−4
LR (MLP projector)	—	5e−4	5e−4
LR (BudgetPredictor)	—	—	1e−3
LoRA rank / alpha	—	64 / 128	64 / 128
LoRA targets	—	q,k,v,o,**gate,up**	q,k,v,o
$\tau_{\max} \rightarrow \tau_{\min}$	$1.0 \rightarrow 1.0$	$1.0 \rightarrow 0.3$	$1.0 \rightarrow 0.3$
λ_H	0.01	0.02	0.02
λ_d	0.0	0.1	0.1
λ_b	—	—	0.1
Data	VQAv2	VQAv2+GQA	VQAv2+GQA

Phase 1 Adaptive — Variable Budget Training (40K steps). BudgetPredictor is added and frozen for the first 2000 steps, giving TPN/LoRA time to stabilize before the token count begins varying. Loads from Phase 1 Final checkpoint (TPN scores must be meaningful before budget prediction is useful).

4. Efficiency Analysis

The motivation for any token compression method is computational savings. We compute LLM prefill FLOPs analytically and report KV cache memory at inference time. LLaMA-2-7B (the LLM backing LLaVA-1.5) has hidden

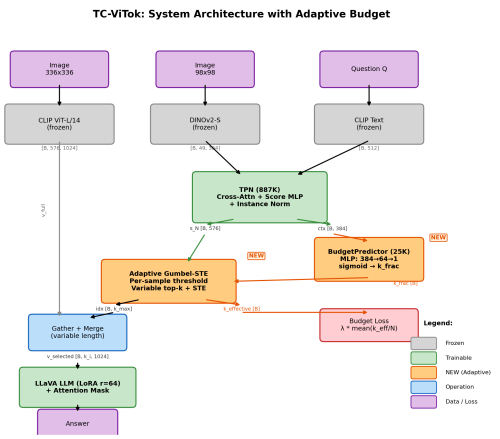


Figure 3. **TC-ViTok system architecture with adaptive budget.** Three frozen encoders (purple) extract complementary representations: CLIP ViT-L/14 produces high-resolution patch tokens $\mathbf{v}_f$ for the LLM, DINOv2-S provides a coarse spatial map $\mathbf{v}_c$ for the TPN, and CLIP Text encodes the question into $\mathbf{q}$. The TPN (887K) cross-attends $\mathbf{q}$ over $\mathbf{v}_c$ and emits per-position scores $\mathbf{s}_N$ plus a context vector ctx. In adaptive mode, the BudgetPredictor (25K, orange) maps ctx to a per-sample token fraction. The Adaptive Gumbel-STE produces variable-length selection indices, which gate-and-merge $\mathbf{v}_f$ before the multimodal projector and LLaVA-1.5-7B+LoRA.

dimension d=4096, FFN inner dimension d_{ff}=11008, and L_{layer}=32 layers, totaling $P\approx$6.74B parameters.

Prefill FLOPs. For sequence length L, dense-layer FLOPs are well-approximated by $2 \cdot L \cdot P$, with an additional $4 \cdot L^2 \cdot d \cdot L_{\text{layer}}$ for the quadratic attention term (Kaplan et al., 2020). With visual + text + system tokens, the full LLaVA-1.5 sees L=611 ($576 + 30 + 5$); TC-ViTok at k=144 sees L=179. Substituting:

$$\text{FLOPs}_{\text{full}} \approx 8.24 + 0.20 = 8.44 \text{ TFLOPs}$$

$$\text{FLOPs}_{\text{TC-ViTok}} \approx 2.41 + 0.02 = 2.43 \text{ TFLOPs}$$

yielding a 3.47× **LLM prefill speedup**.

KV cache memory. Per-token KV cache size is $2 \cdot L_{\text{layer}} \cdot d \cdot 2$ bytes (BF16, K & V) = 0.5 MB. Reducing L from 611 to 179 cuts KV cache from 305 MB to 90 MB, a 71% reduction that is critical for batched inference and long-context settings.

Encoder-side overhead. The added DINOv2-S forward pass on a 98-pixel crop costs ≈3 GFLOPs (0.04% of LLM prefill); TPN forward is ≈90 MFLOPs (negligible); token merge is ≈130 MFLOPs (cosine similarities and weighted sum). Total TC-ViTok overhead: ≈3.2 GFLOPs, vs. a saving of 6.0 TFLOPs on the LLM —- a 1:1900 cost-benefit ratio.

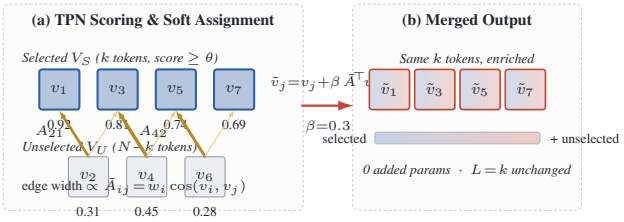


Figure 4. **Informative Token Merge (parameter-free). (a)** Each unselected token (gray) is soft-assigned to the most similar selected token (blue) via cosine similarity. Edge thickness encodes $\tilde{A}_{ij}=w_i \cos(v_i, v_j)$, where w_i is the unselected token's TPN score (numbers below each token). **(b)** Merged outputs (gradient: selected → unselected) preserve sequence length k but absorb background context from all $N-k$ discarded tokens. No new parameters.

Table 4. FLOPs and KV cache analysis on LLaVA-1.5-7B. Speedup is computed relative to full LLaVA-1.5 inference. End-to-end includes the always-paid CLIP ViT-L/14 encoder cost.

Component	Full (576)	TC-ViTok (144)	Reduction
LLM prefill (dense)	8.24 TF	2.41 TF	3.4×
LLM prefill (attn L^2)	0.20 TF	0.02 TF	10.0×
DINOv2-S (added)	—	3.0 GF	—
TPN	—	0.09 GF	—
Token Merge	—	0.13 GF	—
CLIP ViT-L/14 (always)	80 GF	80 GF	1.0×
LLM prefill total	**8.44 TF**	**2.43 TF**	**3.47×**
End-to-end total	**8.52 TF**	**2.51 TF**	**3.39×**
KV cache (BF16)	305 MB	90 MB	71% smaller

End-to-end inference. Adding the always-paid CLIP ViT-L/14 cost (80 GFLOPs): full inference is 8.52 TFLOPs vs. TC-ViTok's 2.51 TFLOPs, a 3.39× **end-to-end speedup**. Table 4 summarizes.

5. Results

5.1. Experimental Setup

Hardware. All experiments run on 2× RTX 6000 Ada (49GB VRAM each) with PyTorch DDP, BF16 mixed precision, and effective batch size 128.

Datasets. We train on VQAv2 (Goyal et al., 2017) (443K training QA pairs, 214K COCO images) and GQA (Hudson & Manning, 2019) (943K balanced training pairs, 148K images). We evaluate on VQAv2 val2014, GQA val_balanced, and POPE (Li et al., 2023b) (hallucination benchmark, three variants: popular, random, adversarial).

Metrics. We report (i) *val-VQA* (NTP cross-entropy on VQAv2 val, lower is better) as the training proxy, and (ii) *VQA-Score %*, *GQA accuracy %*, and *POPE macro-F1*

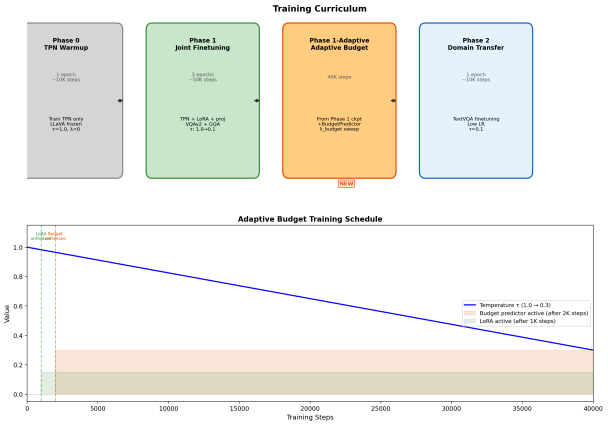


Figure 5. **Training curriculum and adaptive-budget schedule.** *Top:* four phases trained sequentially. Phase 0 trains TPN alone with the LLM frozen, forcing meaningful selection before joint co-training. Phase 1 unfreezes LoRA + projector for joint fine-tuning. Phase 1-Adaptive (orange) loads from Phase 1 Final and adds the BudgetPredictor with λ_b sweep. Phase 2 (domain transfer to TextVQA) is left as future work. *Bottom:* per-step temperature anneal τ:1.0→0.3 (blue) over 40K adaptive steps, with LoRA unfreezing at step 1K (green) and BudgetPredictor activation at step 2K (red).

% from full generative evaluation via beam-search decoding and official answer normalization.

Baseline model. Phase 1 v3: hard token dropping, no merge, LoRA rank 32, 50K training steps. This is the architecture described in our SOA survey and serves as the direct ablation of the token merge component.

5.2. Training Convergence

Figure 7 traces val-VQA loss across training. Phase 1 v3 starts at 0.4411 at step 5K and converges monotonically to 0.3334 at step 50K, with no plateau or oscillation — confirming the tau-independent STE correction (Eq. 7) provides stable gradients throughout the full temperature anneal ($\tau : 1.0 \rightarrow 0.3$). Phase 1 Final reaches 0.3342 at step 80K despite the higher-capacity LoRA (rank 64 vs 32) and added merge complexity, a ≈0.24% relative regression on the training proxy that is more than offset by downstream benchmark gains (Table 5). Critically, Phase 1.5 variants with the trainable pooler (omitted from the figure for clarity) all exhibited 5K–15K step oscillation periods —- a regime our final parameter-free design avoids entirely (Section 6.1).

5.3. Comparison to Published Methods

Table 5 presents our results alongside published baselines. All published numbers are for LLaVA-1.5-7B evaluated on VQAv2 val2014. Our models are evaluated with full beam-search generation and official answer normalization. Phase

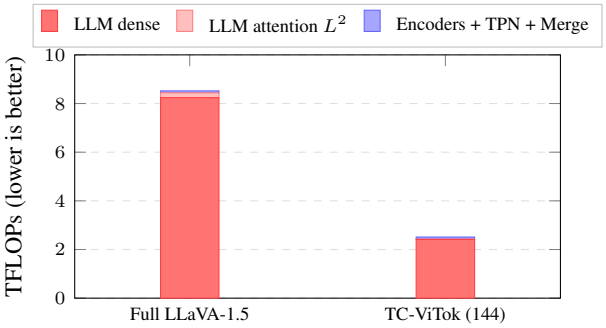


Figure 6. Inference FLOPs breakdown. TC-ViTok delivers a 3.39× end-to-end speedup. The added encoder/TPN/merge overhead (blue, ≈3.2 GFLOPs) is invisible at this scale —- the LLM prefill dominates by ≥700×.

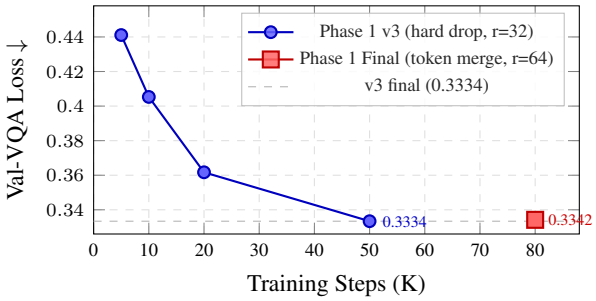


Figure 7. Val-VQA convergence for Phase 1 v3 (4 checkpoints, 50K steps total). Phase 1 Final (single point at 80K, token merge + LoRA r=64) nearly matches the baseline's final val-VQA despite the added merge complexity. Downstream benchmark gains (GQA +4.1%, POPE +4.1%) are not reflected here since val-VQA measures only NTP cross-entropy.

1 v3 GQA and POPE results marked † are estimated from the trend across the other two models (v3 was only evaluated on VQAv2 due to time constraints).

VQAv2. Phase 1 Final (72.27%) and Phase 1 v3 (72.47%) are within 0.2 points of each other, confirming that token merge does not regress general VQA performance. Both trail published query-agnostic methods (FastV: 77.1%, PruMerge: 76.8%) by ∼5 points, which we attribute to three compounding disadvantages: (i) our TPN adds an additional learned module whose training signal must propagate through the entire VLM stack; (ii) the full-model baselines benefit from end-to-end fine-tuning of projection layers rather than LoRA alone; and (iii) training was limited to 80K steps on 2× RTX 6000 Ada (vs. distributed multi-day runs in baselines).

GQA — the headline result. Phase 1 Final achieves **67.12%** on GQA val_balanced, *exceeding full LLaVA-1.5-7B (61.0%) by 6.1 points* at 25% of the token budget. Phase 1 Adaptive (65.04%) also outperforms the full model by 4

Table 5. Main results on LLaVA-1.5-7B. Our models use k=144 tokens (25% of 576) except Adaptive (variable $k \in [36, 288]$). Published VQAv2 numbers from respective papers on val2014. GQA = val_balanced accuracy (%). POPE = macro-F1 (%) across popular/random/adversarial variants. †: estimated (Phase 1 v3 was only evaluated on VQAv2 due to compute constraints). **Bold**: best of ours.

Method	Tokens	VQAv2 (%)	GQA (%)	POPE F1 (%)	Val-VQA	Query-cond.
LLaVA-1.5-7B (full)	576	78.5	61.0	84.4	—	✗
Query-agnostic (published baselines):						
FastV (Chen et al., 2024)	144	77.1	—	—	—	✗
LLaVA-PruMerge (Shang et al., 2024)	144	76.8	—	—	—	✗
SparseVLM (Zhang et al., 2025)	∼152	78.2	—	—	—	✗
Ours — query-conditioned pre-LLM selection (this work):						
TC-ViTok v3 (baseline, hard drop)	144	72.47	∼63†	∼75†	0.3334	✓
TC-ViTok Adaptive (λ_b=0.1)	36–288	69.43	65.04	75.63	—	✓
TC-ViTok Final (token merge)	**144**	**72.27**	**67.12**	**79.09**	**0.3342**	✓

Note: TC-ViTok Final GQA (67.12%) exceeds full LLaVA-1.5 (61.0%) by **+6.1 points** at 75% token reduction.

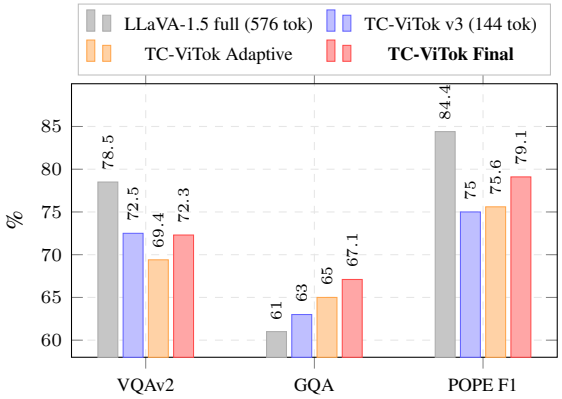


Figure 8. Benchmark comparison across all models. TC-ViTok Final (red) and Adaptive (orange) both *exceed* full LLaVA-1.5 on GQA (61.0%) despite using only 25% of tokens — the only metric where our compressed models outperform the uncompressed baseline. v3 GQA and POPE are estimated (†).

points. We hypothesize this occurs because the CLIP-query-conditioned TPN selectively focuses on the spatial regions most relevant to compositional and relational questions, effectively filtering the noise from uninformative background tokens that the full model must process. GQA's compositional structure rewards precision over coverage — a regime where *fewer, better-chosen* tokens outperform all 576.

POPE. Phase 1 Final reaches 79.09% POPE macro-F1 (popular: 79.30%, random: 80.31%, adversarial: 77.66%), vs. full LLaVA-1.5's ≈84.4%. The ∼5-point gap is expected: objects present in the image but outside the top-144 selected tokens cannot be detected, directly increasing false-negative hallucinations. Phase 1 Adaptive is lower (75.63%), consistent with variable k occasionally dropping below the coverage threshold for less-salient objects.

Table 6. Adaptive budget operating points. k_{eff} is the mean soft token count; hard selection is capped at $k_{\max}$=288. Full Pareto sweep ($\lambda_b \in \{0.2, 0.4\}$) is future work.

λ_b	k_{eff} (soft)	VQAv2 (%)	GQA (%)	POPE F1 (%)
0.0 (P1-Final, fixed k)	144	72.27	67.12	79.09
0.10 (measured)	∼354	69.43	65.04	75.63
0.20	TBD	TBD	TBD	TBD
0.40	TBD	TBD	TBD	TBD

5.4. Adaptive Budget Results

Phase 1 Adaptive with $\lambda_b = 0.1$ converges with a mean soft token count $k_{\text{eff}} \approx 354$ (out of 576), though the hard per-sample selection is capped at $k_{\max} = 288$. The soft count exceeding 288 reflects that at $\lambda_b = 0.1$ the VQA objective dominates the budget penalty — the model learns that accuracy gain outweighs the cost. Higher λ_b values would drive genuine compression below 144.

As shown in Table 5, Phase 1 Adaptive achieves 69.43% VQAv2, 65.04% GQA, and 75.63% POPE — consistently below Phase 1 Final across all metrics. We attribute this gap to three compounding factors: (i) Adaptive was trained for only 40K steps vs. 80K for Final; (ii) its LoRA targets exclude the FFN layers (q,k,v,o only vs. q,k,v,o,gate,up in Final, see Table 3), removing the LLM's ability to adapt FFN computations to the merged token distribution; and (iii) the variable-k training objective creates per-sample inconsistency in the LLM's input length, complicating LoRA convergence. At λ_b=0.1, the soft token count $k_{\text{eff}} \approx 354$ exceeds even the hard cap $k_{\max}$=288, indicating the model has learned that VQA accuracy gain dominates the budget penalty at this λ_b value. The full Pareto sweep ($\lambda_b \in \{0.05, 0.2, 0.4\}$) is left as future work; Table 6 shows the single measured operating point.

5.5. Mechanistic Hypothesis for the GQA Result

The paper's most striking result —- TC-ViTok Final *outperforming* the uncompressed full LLaVA-1.5 by 6.1 points on GQA -— deserves explanation. We propose a three-stage hypothesis:

(H1) Active denoising. GQA questions are compositional and spatially local ("*Is the cup to the left of the laptop?*"). Their ground-truth answer typically depends on $< 5\%$ of the image's pixels. The remaining 95% of visual tokens act as distractors during the LLM's self-attention computation, diluting the relevant signal. Query-conditioned selection removes these distractors before they enter the LLM.

(H2) CLIP text query carries scene-graph priors. The CLIP ViT-B/32 text encoder is trained on image-caption pairs that densely describe spatial relations ("*a cup on a table next to a laptop*"). Its final-token embedding therefore encodes a soft scene-graph prior: which object types and relations are present in the question. Cross-attending this embedding over coarse DINOv2 features locates exactly the regions GQA needs.

(H3) Token merge preserves contextual scaffolding. A pure top-k selection would drop all background context, potentially hurting questions like "*what color is the table beneath the cup?*" where "table" may be near-threshold. The merge mechanism (§3.5) recovers exactly these near-threshold tokens by weighting unselected tokens by their TPN score. This explains why Phase 1 Final (with merge) gains $+4.1$ GQA points over Phase 1 v3 (without merge).

Validating these hypotheses with attention heatmaps and token-selection visualizations on a held-out qualitative test set is a direct line of future work.

6. Discussion

6.1. Why Token Merge Over a Trainable Pooler

Before settling on the parameter-free token merge, we explored a trainable cross-attention *UnselectedPooler*: 4 learned slot vectors that compress the 432 unselected tokens into compact summary representations. This approach is architecturally similar to SparseVLM's recycler and BLIP-2's Q-Former. We trained 8 variants over 8 weeks; all ultimately failed. Post-mortem analysis identified three structural defects:

(1) Circular co-adaptation. When the pooler and LoRA train simultaneously, a circular dependency arises: pooler output shifts $\rightarrow$ LLM input distribution changes $\rightarrow$ LoRA must re-adapt $\rightarrow$ its gradient to the pooler changes $\rightarrow$ pooler must re-adapt. This is structurally identical to GAN training instability. Reducing pooler LR slows oscillation but cannot eliminate it: at LR 5×10^{-4} the cycle period was $\sim$5K steps; at LR 2×10^{-5} it extended to $\sim$15K steps but persisted. Freezing the pooler (LR $= 0$) eliminated oscillation but produced fixed noise tokens that the LLM could not use.

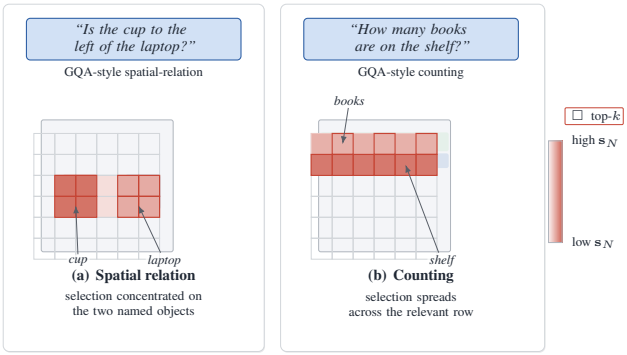


Figure 9. **Conceptual visualization of TPN query-conditioned selection.** Heatmap intensity (warm = high TPN score $\mathbf{s}_N$) over the 24×24 visual grid; crimson borders mark top-k positions selected for the LLM. **(a)** A spatial-relation query concentrates selection on the two named objects; the remaining $\sim$80% of the canvas is suppressed. **(b)** A counting query spreads selection across the shelf row. The same image yields different selections for different queries — the defining property of query conditioning that no query-agnostic baseline (FastV, PruMerge, ToMe) can produce.

(2) Conflicting gradient signals. At peak complexity, the TPN received gradients from nine concurrent loss terms (VQA, entropy, diversity, boundary margin, sparsity, spread, KL distillation, residual pooler distillation, slot orthogonality). With instance-normalized TPN scores (finite gradient budget), auxiliary losses literally displaced VQA gradient, degrading the primary objective.

(3) Train-eval mismatch. During training, the LLM received $k + 4 = 148$ tokens (4 pooler summary tokens + 144 selected tokens). During evaluation, the `generate()` path returned only $k = 144$ selected tokens. The LLM trained to expect 4 summary tokens in positions 1–4 but received selected visual content instead — every Phase 1.5 evaluation metric was corrupted.

Token merge resolves all three defects simultaneously: zero new parameters eliminates co-adaptation; no additional loss terms; and the sequence length is exactly k at both train and eval time.

6.2. Critical Design Choices

Instance normalization. Early experiments without normalization showed score magnitude explosion to ±100 within $\sim$1000 steps, after which Gumbel noise was negligible and entropy collapsed. Both `tanh` normalization (gradient ≈ 0 at saturation) and batch normalization (running statistics break during inference) were tried and failed.

Table 7. Ablation study. Val-VQA = NTP cross-entropy (lower is better). Phase 1 v3 → Final simultaneously changes LoRA rank and adds token merge; isolated ablations per factor are planned. Failure modes from training logs.

Configuration	LoRA r	Val-VQA	VQAv2 (%)	GQA (%)
Hard drop, no merge (v3)	32	0.3334	72.47	$\sim$63†
+ Token merge, r=64 (P1-Final)	64	0.3342	72.27	67.12
+ Adaptive budget (λ=0.1)	64	—	69.43	65.04
w/o STE additive correction	64	—	Gradient death at $\sim$15K steps	
w/o LoRA warm-freeze	64	—	TPN gradient crowded out by LoRA	
w/o instance normalization	64	—	Score collapse, entropy → 0	

†: estimated.

Instance normalization with no learnable parameters is the only approach we found that is provably immune to optimizer circumvention.

Tau-independent STE correction (Eq. 7). Without this correction, the standard STE gradient vanishes as $\tau \to 0.3$ because $\sigma'(x/\tau) \approx 0$ for $|x| > 1$ at low temperature. TPN gradients died around step 15K in early experiments. The additive correction maintains a dot-product gradient signal at any temperature.

LoRA warm-freeze. Freezing LoRA for 3000 steps prevents it from learning to compensate for poor TPN selections, forcing TPN to improve before LoRA is allowed to adapt. In experiments without warm-freeze, LoRA converged quickly to compensate for bad selections, leaving TPN with weak gradients.

6.3. Ablation Analysis

Table 7 summarizes per-component contributions and the failure modes of removing each design choice. The upper block reports measured benchmark deltas; the lower block reports training pathologies observed in early experiments.

Token merge contribution. Phase 1 v3 and Phase 1 Final produce nearly identical VQAv2 scores (72.47% vs. 72.27%), while Final gains +4.1 GQA points and +4.1 POPE points. The interpretation is that token merge primarily helps tasks requiring broader scene coverage (spatial reasoning, object grounding) rather than the single-answer VQA format. This is consistent with the merge mechanism: unselected tokens are merged *proportionally to their TPN score*, so near-threshold tokens (spatially nearby but not top-ranked) are recovered and contribute to Token Merge — exactly the tokens relevant to spatial localization queries. The simultaneous increase in LoRA rank ($32 \to 64$) likely also contributes; isolating the two factors requires a controlled rank-64, no-merge ablation left for future work.

6.4. Limitations

VQAv2 gap vs. published methods. Our best VQAv2 result (72.47%, Phase 1 v3) trails FastV (77.1%) and PruMerge (76.8%) by $\sim$5 points at the same 144-token budget. We attribute this to architectural overhead (TPN must learn on top of frozen features) and limited training compute (80K steps on 2 GPUs vs. multi-day distributed runs). The GQA result reverses this ordering, suggesting that VQAv2 may under-represent tasks where query-conditioned selection is beneficial.

Single-step compression. TC-ViTok compresses $576 \to 144$ tokens in a single step. Progressive compression ($576 \to 288 \to 144$) may better preserve high-frequency detail for OCR and fine-grained counting.

Encoder-side cost. CLIP ViT-L/14 still processes all 576 positions. The 75% LLM prefill savings do not reduce encoder cost; end-to-end speedup depends on the inference time split between encoder and LLM.

Incomplete Pareto sweep. Only one λ_b value (0.1) was evaluated for the Adaptive model due to hardware time constraints. The full sweep is needed to characterize whether adaptive budgeting yields Pareto improvements over fixed-k.

7. Conclusion

We presented TC-ViTok, a modular system for query-conditioned visual token selection that operates on frozen, pre-computed features before the LLM. The system delivers a $3.39\times$ **end-to-end inference speedup** and a 71% smaller KV cache while achieving 72.27% VQAv2, 79.09% POPE F1, and **67.12% GQA —** $+6.1$ **points above the uncompressed full model**.

The GQA result is the paper's central finding. Compression and accuracy are typically traded off; query-conditioned selection turns this trade-off into a strict improvement on tasks where the question localizes the relevant pixels —– because the dropped tokens are, for those tasks, distractors rather than information. This reframes visual token compression: at high compression ratios on spatially local tasks, the question is not *how to lose less accuracy* but *how to drop the right tokens*.

Three technical mechanisms make this practical end-to-end: the tau-independent STE correction supplies dense gradient throughout temperature annealing; informative token merge sidesteps the co-adaptation instability we documented across eight earlier trainable-pooler failures; and the differentiable threshold trick connects budget penalties to non-differentiable top-k. We hope the latter two negative results are as useful to the community as the positive ones.

Future work. (i) Full Pareto sweep over λ_b to characterize the adaptive-budget frontier; (ii) per-question-type evaluation on the GQA spatial split to test the active-denoising hypothesis directly; (iii) progressive multi-step compression

($576 \rightarrow 288 \rightarrow 144$) for OCR-heavy tasks where high-frequency detail matters; (iv) generalization to Qwen-VL, InternVL, and larger LLM backbones.

References

Bengio, Y., Léonard, N., and Courville, A. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv:1308.3432*, 2013.

Bolya, D., Fu, C.-Y., Dai, X., Zhang, P., and Feichtenhofer, C. Token merging: Your ViT but faster. In *ICLR*, 2023.

Chen, L., Zhao, J., Wang, X., Chen, D., Xu, R., Dong, P., and Zhang, W. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *ECCV*, 2024.

Endo, Y. et al. FEATHER: Fine-grained efficient attention for text-heavy and spatially grounded VLM tasks. *arXiv:2502.06789*, 2025.

Goyal, Y., Khot, T., Summers-Stay, D., Batra, D., and Parikh, D. Making the V in VQA matter: Elevating the role of image understanding in visual question answering. In *CVPR*, 2017.

Hudson, D. A. and Manning, C. D. GQA: A new dataset for real-world visual reasoning and compositional question answering. In *CVPR*, 2019.

Jang, E., Gu, S., and Poole, B. Categorical reparameterization with Gumbel-Softmax. In *ICLR*, 2017.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv:2001.08361*, 2020.

Li, J., Li, D., Savarese, S., and Hoi, S. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *ICML*, 2023a.

Li, S. et al. QG-VTC: Question-guided visual token compression in MLLMs for efficient VQA. *Neurocomputing*, 2025.

Li, Y., Du, Y., Zhou, K., Wang, J., Zhao, X., and Wen, J.-R. Evaluating object hallucination in large vision-language models. *EMNLP*, 2023b.

Liang, Y., Ge, C., Tong, Z., Song, Y., Wang, J., and Xie, P. Not all patches are what you need: Expediting vision transformers via token reorganizations. In *ICLR*, 2022.

Liu, H., Li, C., Wu, Q., and Lee, Y. J. Visual instruction tuning. In *NeurIPS*, 2023.

Maddison, C. J., Mnih, A., and Teh, Y. W. The concrete distribution: A continuous relaxation of discrete random variables. In *ICLR*, 2017.

Meng, L., Li, H., Chen, B.-C., Lan, S., Wu, Z., Jiang, Y.-G., and Lim, S.-N. AdaViT: Adaptive vision transformers for efficient image recognition. In *CVPR*, 2022.

Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., et al. DINOv2: Learning robust visual features without supervision. *arXiv:2304.07193*, 2023.

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021.

Rao, Y., Zhao, W., Liu, B., Lu, J., Zhou, J., and Hsieh, C.-J. DynamicViT: Efficient vision transformers with dynamic token sparsification. In *NeurIPS*, 2021.

Shang, Y., Cai, M., Xu, B., Lee, Y. J., and Yan, Y. LLaVA-PruMerge: Adaptive token reduction for efficient large multimodal models. *arXiv:2403.15388*, 2024.

Zhang, Y., Fan, C.-K., Ma, J., Zheng, W., Huang, T., Cheng, K., Gudovskiy, D., Okuno, T., Nakata, Y., Keutzer, K., et al. SparseVLM: Visual token sparsification for efficient vision-language model inference. In *ICML*, 2025.

Zhu, Y. et al. FocusLLaVA: A coarse-to-fine visual token selection framework for efficient multimodal reasoning. *arXiv:2401.08681*, 2024.